Intro
Conceptos básicos
Propiedades del contenedor flexible
flex-direction
justify-content
align-items
flex-wrap
flex-flow
align-content
Propiedades de los elementos flexibles
order
flex-grow
flex-shrink
flex-basis
flex
align-self
Demostración interactiva
Usos frecuentes de flexbox
Centrado vertical del texto de un elemento
Centrado horizontal y vertical de un elemento hijo dentro de su padre
Alineación de título y subtítulo
Columnas de igual altura
Menú de navegación con un número indeterminado de opciones
Menú en acordeón
Galería de imágenes
Pie pegajoso
El Santo Grial
Huecos entre elementos
Barra de navegación
Referencias

Intro

La propiedad *flexible box*, o *flexbox* (caja flexible), es un nuevo modelo de formato visual de CSS3 que permite colocar los elementos de una página para que se comporten de forma predecible cuando el diseño de la página debe acomodarse a diferentes tamaños de pantalla y diferentes dispositivos.

Los elementos hijos de una caja flexible pueden **colocarse en cualquier dirección** y pueden tener dimensiones flexibles para adapterse al espacio visible. Además, **el orden de visualización de los elementos es independiente** del orden que estos tengan en el código fuente.

Lo que caracteriza un diseño flexible es su habilidad para alterar el ancho y alto de sus elementos para ajustarse lo mejor posible al espacio disponible en cualquier dispositivo. Un contenedor flexible expande sus elementos para rellenar el espacio libre, o los comprime para evitar que rebasen el área prevista.

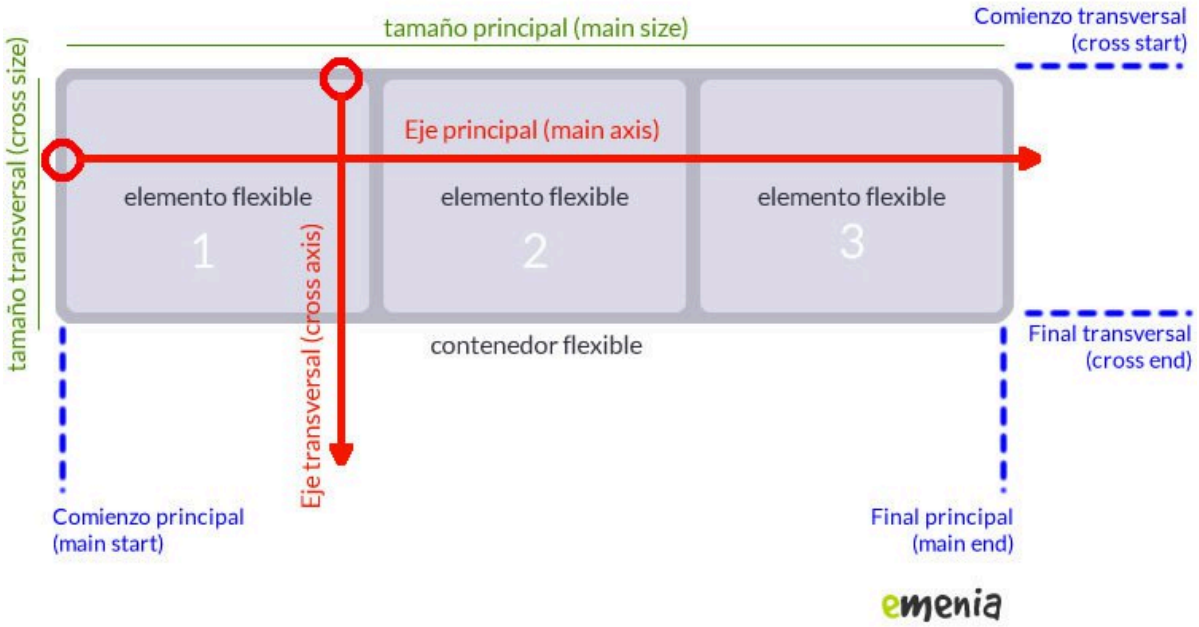
El algoritmo del modelo de diseño de cajas flexibles no parte de niguna dirección predeterminada, al contrario de lo que ocurre con el modelo de bloque, que asume una disposición vertical de los elementos, o lo que pasa con el modelo "en línea", que asume una disposición horizontal.

El modelo de cajas flexibles es más apropiado para diseños de pequeña escala, mientras que el modelo de rejilla (`grid`),es adecuado para diseños de gran escala.

Conceptos básicos

En el modelo en bloque, los elementos hijos se organizan en un único eje vertical, mientras que en el modelo en línea lo hacen en un único eje horizontal. Sin embargo, el modelo de caja flexible trabaja con ambos ejes, el vertical y el horizontal, y nos permite especificar cuál de ellos es el principal (*main axis*) y cuál el secundario (*cross axis*). La elección de cuál es el eje principal es importante, ya que afectará al significado del resto de propiedades.

Por ejemplo, en el siguiente diagrama se muestra un contenedor flexible cuyo eje principal es el eje horizontal, lo que implica que las cajas hijas de la caja flexible se muestren una a continuación de la otra horizontalmente. En este caso la dirección dentro del eje principal es de izquierda a derecha.



Para comprender bien el modelo de cajas flexibles es necesario definir una serie de conceptos básicos:

- **Contenedor flexible** (*flex container*): Elemento padre al que se le establece el modelo de cajas flexibles mediante `display: flex` o `display: inline-flex`.
- **Elemento flexible** (*flex item*): Cada elemento hijo de un contenedor flexible. **Si el contenedor flexible incluye texto sin marcar, éste se envuelve automáticamente en un elemento flexible anónimo.**
- Ejes: El diseño de caja flexible sigue dos ejes:
    - **El eje principal** (*main axis*) es el eje a lo largo del cual los elementos flexibles se suceden unos a otros. Se establece mediante la propiedad `flex-direction`. Por su parte, la propiedad `justify-content` define cómo los elementos flexibles se disponen a lo largo del eje principal en la línea en curso.

    - **El eje secundario** (*cross axis*) es el eje perpendicular al eje principal. La propiedad `align-items` define cómo los elementos flexibles se disponen a lo largo del eje secundario de la línea en curso. Por su parte, la propiedad `align-self` define cómo cada elemento flexible se alinea respecto al eje secundario, y sustituye al valor por defecto establecido por `align-items`.
- **Direcciones**: Dentro de cada eje se debe conocer la dirección en la que deben disponerse los elementos flexibles, indicando cual es el punto inicial en el eje principal (*main start* y *main end*, respectivamente) y en el eje secundario (*cross start* y *cross end*). Por otro lado, la propiedad `order` asigna elementos a grupos ordinales y determina qué elementos aparecen primero dentro del eje principal, independientemente de su posición en el documento HTML.
- **Líneas**: Los elementos flexibles pueden disponerse en una sola o varias líneas de acuerdo con la propiedad `flex-wrap`, que controla la dirección del eje secundario y la dirección en la que las nuevas líneas se apilan en dicho eje.
- **Dimensiones**: El tamaño (anchura / altura) de los elementos flexibles se denomina tamaño principal (*main size*), relativo al eje principal, y tamaño secundario (*cross size*), relativo al eje secundario. Así, las propiedades `min-height` y `min-width` incorporan un nuevo valor, denominado `auto`, que establece el tamaño mínimo de un elemento felxible. Por su parte, las propiedades `flex-basis`, `flex-grow`, y `flex-shrink` para establecer el grado de flexibilidad en el tamaño de los elementos flexibles, de manera que éstos puedan crecer o encogerse.

Propiedades del contenedor flexible

flex-direction

La propiedad `flex-direction` define cuál es el eje principal del contenedor flexible y la dirección de dicho eje.

Propiedad	flex-direction
Valores	row | row-reverse | column | column-reverse
Se aplica a	Contenedores flexibles
Valor inicial	row
Descripción	Define cuál es el eje principal del contenedor flexible y la dirección de dicho eje.

El significado de los valores es el siguiente:

- `row`: El eje principal del contenedor flexible es el eje horizontal y su dirección es de izquierda a derecha.
- `row-reverse`: El eje principal del contenedor flexible es el eje horizontal y su dirección es de derecha a izquierda.
- `column`: El eje principal del contenedor flexible es el eje vertical y su dirección es de arriba a abajo.
- `column-reverse`: El eje principal del contenedor flexible es el eje vertical y su dirección es de abajo a arriba

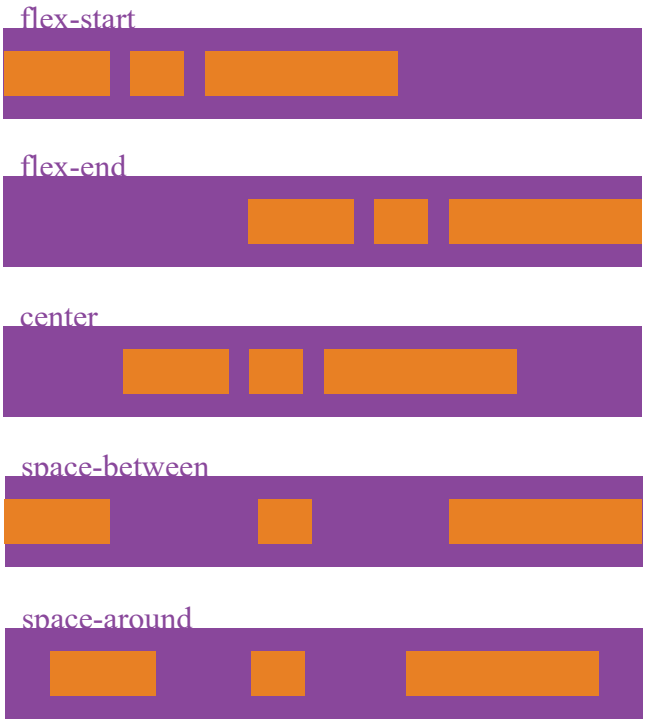
justify-content

La propiedad `justify-content` define cómo el navegador distribuye el **espacio sobrante** en las líneas del **eje principal** en la que se muestran los elementos flexibles. El espacio sobrante es calculado después de aplicar el tamaño y los márgenes de los elementos flexibles, y siempre y cuando NO se haya establecido que alguna caja crezca para ocupar dicho espacio sobrante mediante la propiedad `flex-grow`.

Propiedad	**justify-content**
Valores	flex-start | flex-end | center | space-between | space-around
Se aplica a	Contenedores flexibles
Valor inicial	flex-start
Descripción	Define cómo el navegador distribuye el espacio sobrante en las líneas del eje principal.

El significado de los valores es el siguiente:

- `flex-start`: Los elementos flexibles se sitúan en la línea comenzando desde el punto inicial del eje principal (*main start*), uno seguido del otro sin espacio intermedio adicional, por lo que todo el espacio sobrante se sitúa en el otro extremo del eje principal (al lado del *main end*).
- `flex-end`: Los elementos flexibles se sitúan en la línea comenzando desde el punto final del eje principal (*main end*), uno seguido del otro sin espacio intermedio adicional, por lo que todo el espacio sobrante se sitúa en el otro extremo del eje principal (al lado del *main start*).
- `center`: Los elementos flexibles se sitúan centrados en la línea del eje principal, uno seguido del otro sin espacio intermedio adicional, por lo que todo la mitad del espacio sobrante se sitúa en cada extremo del eje principal (al lado del *main start* y al lado del *main end*).
- `space-between`: Los elementos flexibles se distribuyen uniformemente sobre la línea del eje principal. El espaciamiento se hace de tal manera que el espacio adyacente entre dos elementos es el mismo. Los puntos inicial principal y final principal se alinean, respectivamente, con el borde del primer y último elemento flexible.
- `space-around`: Los elementos flexibles se alinean uniformemente de tal manera que **el espacio entre dos elementos adyacentes es el mismo**. **El espacio vacío anterior al primer elemento y posterior al último elemento equivale a la mitad del espacio entre dos elementos adyacentes.**



align-items

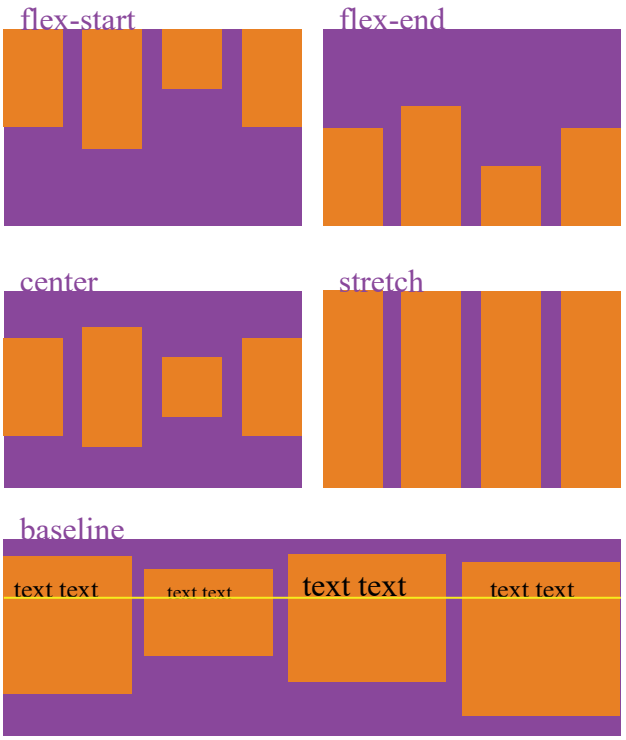
La propiedad `align-items` define cómo el navegador distribuye el **espacio sobrante dentro de las líneas del eje secundario** en las que se muestran los elementos flexibles. El espacio sobrante es calculado después de aplicar el tamaño y los márgenes de los elementos flexibles.

Propiedad	**align-items**
Valores	flex-start | flex-end | center | baseline | stretch
Se aplica a	Contenedores flexibles
Valor inicial	stretch
Descripción	Define cómo el navegador distribuye el espacio sobrante en las líneas del eje principal.

El significado de los valores es el siguiente:

- `flex-start`: Los elementos flexibles se alinean con el punto inicial del eje secundario (*cross start*) dentro de la línea.
- `flex-end`: Los elementos flexibles se alinean con el punto final del eje secundario (*cross end*) dentro de la línea.
- `center`: Los elementos flexibles son centrados dentro de la línea sobre su eje secundario (*cross axis*) . Si el tamaño transvesal (*cross size*) del elemento es mayor al del contenedor, se excederá por igual hacia ambas direcciones.
- `baseline`: Los elementos flexibles son situados de modo que sus bases queden alienadas dentro de la línea sobre su eje secundario (*cross axis*). El elemento con la distancia mayor entre su límite transversal inicial y su base es alineado con el punto final de la línea del eje secundario (*cross end*), y el resto de elementos se sitúan de manera para que sus bases queden alineadas con la base de dicho elemento.

- `stretch`: Las elementos flexibles son estirados de modo que su tamaño transversal (*cross size*) sea el mismo de la línea, manteniendo sus restricciones de anchura y altura.



-

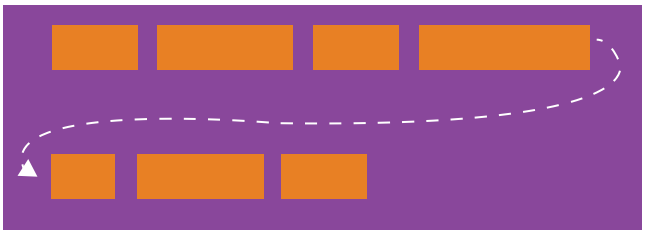
flex-wrap

La propiedad `flex-wrap` especifica si los elementos flexibles son obligados a permanecer en una misma línea o pueden fluir en varias líneas (*wrap*) en el eje principal. En este último caso también podremos especificar la dirección en la cual serán apilados las líneas dentro del eje secundario.

Propiedad	**flex-wrap**
Valores	nowrap | wrap | wrap-reverse
Se aplica a	Contenedores flexibles
Valor inicial	nowrap
Descripción	Define si los elementos flexibles pueden fluir en varias líneas dentro del eje principal, y la dirección de las mismas dentro del eje secundario.

El significado de los valores es el siguiente:

- `nowrap`: Los elementos flexibles son distribuidos en una sola línea, lo cual puede llevar a que se desborde el contenedor flexible.
- `wrap`: Los elementos flexibles son colocados en varias líneas. Las líneas se apilan desde el punto inicial hasta el punto final del eje secundario.
- `wrap-reverse`: Los elementos flexibles son colocados en varias líneas. Las líneas se apilan desde el punto final hasta el punto inicial del eje secundario.



-

flex-flow

La propiedad `flex-flow` es una propiedad *shorthand* de las propiedades individuales `flex-direction` y `flex-wrap`.

Propiedad	flex-flow
Valores	
Se aplica a	Contenedores flexibles
Valor inicial	row nowrap
Descripción	-

align-content

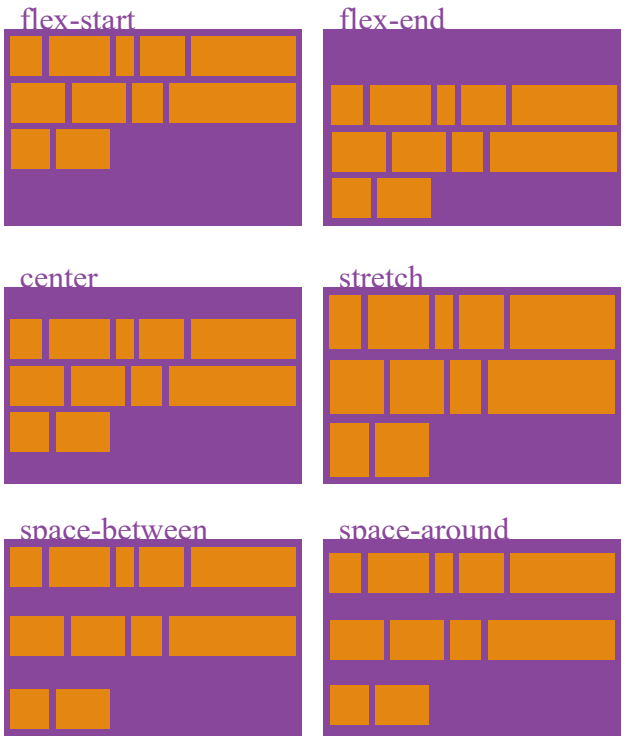
La propiedad `align-content` define cómo el navegador distribuye el **espacio sobrante en el eje secundario no utilizado por las líneas** en las que se muestran los elementos flexibles. El espacio sobrante es calculado después de aplicar el tamaño y los márgenes de los elementos flexibles, y disponerlos en una o varias líneas a lo largo del eje principal.

Propiedad	align-content
Valores	flex-start | flex-end | center | space-between | space-around | stretch
Se aplica a	Contenedores flexibles
Valor inicial	stretch
Descripción	Define cómo el navegador distribuye el espacio sobrante en eje secundario.

El significado de los valores es el siguiente:

- `flex-start`: Las líneas son ajustadas a partir del punto de inicio del eje secundario. El borde transversal de inicio de la primera línea y el del contenedor flexible quedan unidos. Cada línea siguiente es unida a su predecesora.
- `flex-end`: Las líneas son ajustadas a partir del punto final del eje transversal. El borde transversal final de la última línea y el del contenedor flexible quedan unidos. Cada línea que precede es unida a la línea siguiente.
- `center`: Las líneas son ajustadas hacia el centro del contenedor flexible. Las líneas son unidas entre sí, y centradas dentro del contenedor. El espacio entre el borde transversal de inicio y la primera línea, y el que hay entre el borde transversal final y la última línea es el mismo.

- `space-between`: Las líneas son distribuidas de manera uniforma en el contenedor flexible. El espaciado se hace de modo que la separación entre cualquier par de elementos adyacentes sea el mismo. Los bordes transversales de inicio y de fin del contenedor son unidos a los bordes de la primera y última línea, respectivamente.
- `space-around`: Las líneas son distribuidas uniformemente de modo que el espacio entre cualquier par de elementos adyacentes sea el mismo. El espacio vacío antes de la primera línea y el espacio después de la última es igual a la mitad del espacio entre cualquier par de líneas adyacentes.
- `stretch`: Las líneas son estiradas para usar el espacio sobrante. El espacio libre en el contenedor es dividido por igual entre todas las líneas.



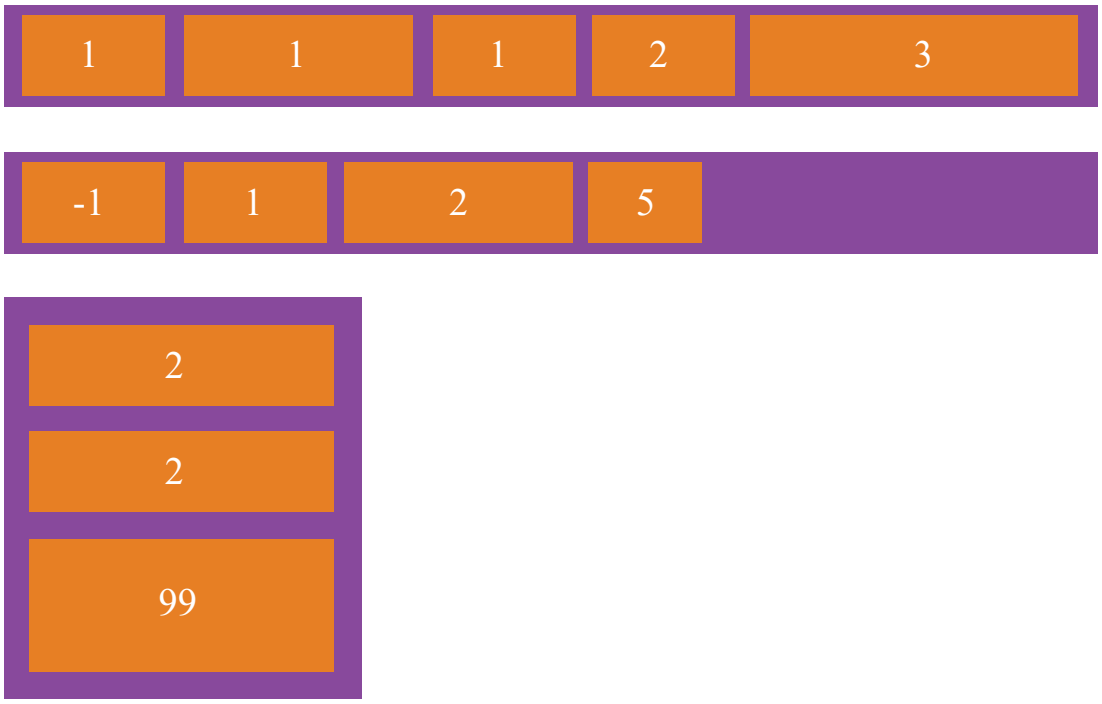
-

Propiedades de los elementos flexibles

order

La propiedad `order` especifica el orden utilizado para disponer los elementos en su contenedor flexible. Los elementos estarán dispuestos en orden ascendente según el valor de esta propiedad. Los elementos con el mismo valor se dispondrán en el orden en el cual aparecen en el documento HTML.

Propiedad	**order**
Valores	
Se aplica a	Elementos flexibles
Valor inicial	0
Descripción	Especifica el orden de un elemento dentro de su contenedor flexible.

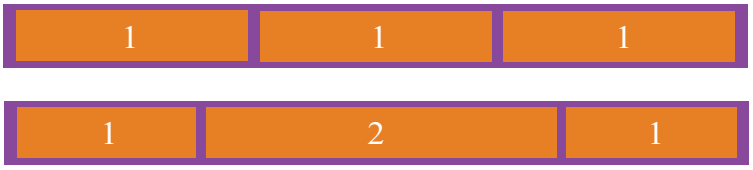


flex-grow

La propiedad `flex-grow` define la habilidad de un **elemento flexible de poder crecer en el eje principal** si hay espacio disponible en la línea en la que está situado. Acepta un valor numérico sin unidad que actúa como proporción respecto a la suma de los valores de esta propiedad de todos los elementos flexibles situados en la misma línea del contenedor flexible. El espacio disponible dentro de la línea se repartirá entre los elementos flexibles que contenga, atendiendo a la proporción especificada por cada uno de ellos.

Vemos un ejemplo: supongamos que tenemos cuatro elementos flexibles en una línea del eje principal del contenedor flexible, y dos de ellos tienen `flex-grow: 0` (valor por defecto) y otros dos `flex-grow: 1`, entonces los dos primeros no recibirán nada del espacio sobrante del eje principal dentro de la línea, mientras que los otros dos recibirán cada uno la mitad de dicho espacio sobrante, debido a la proporción `1 / 1 + 1`. Si al cuarto elemento se le asigna `flex-grow: 2`, entonces el tercer elemento recibirá un tercio del espacio sobrante, `1 / 1 + 2` y el cuarto elemento dos tercios `2 / 1 + 2`.

Propiedad	**flex-grow**
Valores	<número>
Se aplica a	Elementos flexibles
Valor inicial	0
Descripción	Define la proporción de espacio sobrante dentro del eje principal en la que crecerá el elemento.



flex-shrink

La propiedad `flex-shrink` determina el **factor de decrecimiento de un elemento flexible si el conjunto de elementos flexibles no cabe en la línea del eje principal del contenedor flexible.** Acepta un valor numérico sin unidad que actúa como proporción respecto a los valores de esta propiedad de todos los elementos flexibles situados en la misma línea del contenedor flexible. **El valor por defecto para esta propiedad es 1, por lo que los elementos se encogerán en la misma proporción. Sin embargo, si para un elemento establecemos** `flex-shrink: 0`, dicho elemento no se encogerá.

Propiedad	**flex-shrink**
Valores	<número>
Se aplica a	Elementos flexibles
Valor inicial	1
Descripción	Define el factor de decrecimiento de un elemento flexible dentro de la línea del eje principal del contenedor flexible cuando todos sus elemento no caben en ella.

flex-basis

La propiedad `flex-basis` **especifica el tamaño inicial de un elemento flexible en el eje principal, antes de que se reparta el espacio sobrante de la línea en la que se encuentre.**

Propiedad	**flex-basis**
Valores	| auto
Se aplica a	Elementos flexibles
Valor inicial	auto
Descripción	Define el tamaño inicial de un elemento flexible en el eje principal, antes de que se reparta el espacio sobrante de la línea en la que se encuentre.

Analicemos los posibles valores:

- La longitud corresponderá a un número seguido de una unidad, o a un **porcentaje del tamaño principal del contenedor flexible padre. Los valores negativos no son válidos.**
- El valor `auto` significa el valor de la propiedad `width` o `height`, dependiendo de cuál sea el eje principal.

flex

La propiedad CSS flex es una propiedad *shorthand* que especifica la capacidad de un elemento flexible para alterar sus dimensiones para llenar el espacio disponible. Los elementos flexibles pueden ser estirados para utilizar el espacio disponible proporcional a su factor de crecimiento flexible o su factor de contracción flexible para evitar desbordamiento.

Propiedad	flex
Valores	[] | none
Se aplica a	Elementos flexibles
Valor inicial	0 1 auto
Descripción	Especifica la capacidad de un elemento flexible para alterar sus dimensiones para llenar el espacio disponible en la línea del eje principal

Los valores para `flex-shrink` y `flex-basis` son opcionales. El valor `none` se traduce a `0 0 auto`.

Se recomienda el uso de esta propiedad "shorthand" en vez de usar las propiedades individuales.

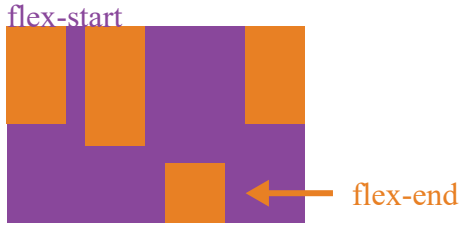
align-self

La propiedad `align-self` permite alinear un elemento flexible dentro de la línea respecto al eje secundario, sobrescribiendo el comportamiento definido por la propiedad `align-items` del contenedor flexible.

Propiedad	align-self
Valores	flex-start | flex-end | center | baseline | stretch | auto
Se aplica a	Contenedores flexibles
Valor inicial	auto
Descripción	Alinea un elemento flexible dentro de la línea respecto al eje secundario, sobrescribiendo el comportamiento definido por la propiedad `align-items` del contenedor flexible

El valor `auto` indica que se use lo especificado por la propiedad `align-items` del contenedor flexible. El resto de valores tienen el mismo significado que en dicha propiedad.

Debemos tener en cuenta que las propiedades `float`, `clear` y `vertical-align` NO tienen efecto sobre los elementos flexibles.



Demostración interactiva

En el siguiente enlace puede ver una impresionante demostración interactiva de las propiedades anteriores

> Demostración interactiva de flexbox

Usos frecuentes de flexbox

Centrado vertical del texto de un elemento

La técnica consiste en **establecer el elemento con contenedor flexible y usar `align-items: center` para centrar el texto**. Esta técnica funciona porque el texto contenido en un contenedor flexible es considerado un elemento flexible como cualquier otro a la hora de posicionarlo.

La ventaja de esta técnica frente a la de establecer `line-height` a la misma altura de la caja es que funciona incluso si el texto tiene varias líneas.

```
.con-texto-centrado-verticalmente {
  display: flex;
  align-items: center;
}
```

Veamos una demostración:

See the Pen Centrado vertical de texto con flexbox by Pedro Joya (@pedrojoya) on CodePen.

```
<div class="padre">Soy un texto centrado verticalmente aunque tenga varias líneas</div>
```

```
.padre {
  padding: 10px;
  font-size: 20px;
  width: 25%;
  margin: 0 auto;
  height: 200px;
  background-color: beige;
  border-radius: 5px;
  border: solid 2px brown;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

> Demo de centrado vertical de texto multilínea con flexbox

Centrado horizontal y vertical de un elemento hijo dentro de su padre

La técnica consiste en establecer en el elemento padre las siguientes propiedades:

```
.centrados {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Veamos una demostración:

See the Pen Flexbox: centrado vertical de un elemento dentro de su padre by Pedro Joya (@pedrojoya) on CodePen.

```
<div class="padre centrados">
  <div class="titulo">Caja padre</div>
  <div class="caja">Caja hija</div>
</div>
```

```
.padre {
  background-color: #B5A8BE;
  width: 50%;
  height: 200px;
  margin: 1rem auto;
  position: relative;
}
.centrados {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  align-items: center;
}
.titulo {
  position: absolute;
  text-align: right;
  color: white;
  top: .5rem;
  right: .5rem;
}
.caja {
  padding: 1.5rem;
  color: white;
  background-color: #715683;
  border: 2px solid #4B2961;
}
```

> Demo de centrado doble de un elemento con flexbox

Alineación de título y subtítulo

La técnica consiste en establecer en un elemento padre de título y subtítulo las siguientes propiedades:

```
.titulo {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: flex-end;
}
```

Veamos una demostración:

See the Pen Flexbox: Alineación de título y subtítulo by Pedro Joya (@pedrojoya) on CodePen.

```
<div class="padre titulo">
  <h2>No te digo trigo</h2>
  <h4>por no llamarte Rodrigo</h4>
</div>
```

```
.padre {
  width: 80%;
  margin: 1rem auto;
  border-bottom: 2px solid #ccc;
}
.titulo {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: flex-end;
}
h2, h4 {
  margin: .1rem .5rem 0 .5rem;
}
h2 {
  color: rgba(0, 0, 0, .8);
}
h4 {
  color: rgba(0, 0, 0, .5);
}
```

> Demo de alineación de título y subtítulo con flexbox

Columnas de igual altura

Por defecto, los elementos flexibles dentro de la misma línea del eje principal del contenedor flexible **se estiran para ocupar todos el mismo tamaño en el eje secundario**, debido a que, por defecto, la propiedad `align-items` del contenedor flexible tiene el valor `stretch`. **Podemos aprovechar este valor para crear dentro de un elemento contenedor columnas de la misma altura, independientemente de la altura real del contenido de cada columna**. Veamos una demostración:

See the Pen Flexbox: Columnas de igual altura by Pedro Joya (@pedrojoya) on CodePen.

```
<ul class="columnas">
  <li>
    <h2>Columna 1</h2>
```

```
    <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Vel eum, delectus minima possimus veniam dignissimos omnis ducimus dolore ipsum adipisci quo placeat laudantium, iusto, repudiandae inventore error atque</p>
  </li>
  <li>
    <h2>Columna 2</2>

  </li>
  <li>
    <h2>Columna 3</h2>
    <p>Impedit sequi perferendis suscipit quidem, illum nemo reprehenderit eveniet quod autem quaerat quo, ipsum quam provident</p>
  </li>
  <li>
    <h2>Columna 4</h2>
    <p>Cupiditate non animi vero, voluptatem, dolorem sapiente. Mollitia libero, ab ex iste, saepe voluptatem earum, labore dicta temporibus ducimus nostrum cumque rerum voluptatibus. </p>
  </li>
  <li>
    <h2>Columna 5</h2>
    <p>Hic velit quis nihil ex quae aut eum dolor commodi quod perspiciatis fugit labore eveniet totam quidem ratione voluptates rerum perferendis sunt natus, minima tempora ipsum neque voluptatem. Perspiciatis, velit.</p>
    <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Quisquam illo, modi voluptate blanditiis quis minima sit, autem nisi ut amet quae! Quod explicabo illo delectus vitae, perspiciatis beatae quasi magni.</p>
  </li>
</ul>
```

```
*, *:before, *:after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
html {
  --num-columnas: 3;
  --margen-columna: 1rem;
}
ul {
  list-style-type: none;
}
ul.columnas {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
}
li {
  padding: 1rem;
  margin: var(--margen-columna);
  background: #dedede;
}
.columnas li {
  flex: 1 1 calc((100% - 2 * var(--margen-columna) * var(--num-columnas)) / var(--num-columnas));
```

```
}
h2, p {
  margin-bottom: .5rem;
}
```

> Demo de columnas de igual altura con flexbox

Menú de navegación con un número indeterminado de opciones

Como vimos en un apartado anterior, para crear un **menú horizontal** existían varias técnicas, como por ejemplo hacer uso de la **propiedad** `display: inline-block` **en el elemento** `<ul>` **o hacer que los elementos** `<li>` **flotaran dentro del** `<ul>` **gracias a la propiedad** `float:left`**.** En ambas técnicas, si queríamos que todos las opciones se mostraran con el mismo ancho **era necesario conocer de antemano el número de opciones de las que iba a disponer el menú,** para hacer el cálculo del porcentaje a asignar como valor de la propiedad `width` de los elementos `<li>`.

Gracias a *flexbox* ya no es necesario conocer este dato, ya que podremos disponer el `<ul>` **como un contenedor flexible cuyos elementos flexibles** `<li>` **adaptan su anchura dependiendo del número de opción correspondientes. El secreto está en establecer para los elementos flexibles que la anchura de referencia sea 0, mediante** `flex-basis: 0`**, y luego hacer que crezcan proporcionalmente en base al espacio sobrante, mediante** `flex-grow: 1`**.** Veamos una demostración:

See the Pen Flexbox: Menú horizonal número indeterminado anchura igual by Pedro Joya (@pedrojoya) on CodePen.

```
<nav>
  <ul class="ancho-fijo">
    <li><a href="#" title="Inicio">Inicio</a></li>
    <li><a href="#" title="Blog">Blog</a></li>
    <li><a href="#" title="Trabajo">Trabajo y estudios varios</a></li>
    <li><a href="#" title="Recursos">Recursos</a></li>
    <li><a href="#" title="Acerca de">Acerca de</a></li>
  </ul>
</nav>
```

```
*, *:before, *:after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
nav {
  margin: 1rem;
}
nav ul {
  list-style: none;
  border-bottom: solid 1px rgba(0,0,0, .3);
}
nav ul li {
  transition: all .5s;
  display: flex;
```

```
  justify-content: center;
  align-items: center;
}
nav ul li a {
  display: block;
  padding: .5rem;
  text-align: center;
  text-decoration: none;
  color: rgba(0, 0, 0, .8);
}
nav ul li:hover {
  background-color: #8E799C;
  box-shadow: 0 -8px 0 #5A3B6E inset;
}
nav ul li:hover a {
  color: white;
}
nav ul.ancho-fijo {
  display: flex;
  flex-wrap: wrap;
}
nav ul.ancho-fijo li {
  flex: 1 1 0;
}
```

> Demo de menú de navegación con ítems de igual anchura mediante flexbox

Si lo que queremos es que cada opción tenga una anchura proporcional a su texto, haremos que los elementos flexibles tomen como referencia de **anchura la correspondiente a su contenido, con** `flex-basis: auto`, y luego haremos que crezcan proporcionadamente en base al espacio sobrante, mediante `flex-grow: 1`. Veamos una demostración:

See the Pen Flexbox: Menú horizonal número indeterminado anchura proporcional by Pedro Joya (@pedrojoya) on CodePen.

```
<nav>
  <ul class="proporcional">
    <li><a href="#" title="Inicio">Inicio</a></li>
    <li><a href="#" title="Blog">Blog</a></li>
    <li><a href="#" title="Trabajo">Trabajo y estudios varios</a></li>
    <li><a href="#" title="Recursos">Recursos</a></li>
    <li><a href="#" title="Acerca de">Acerca de</a></li>
  </ul>
</nav>
```

```
*, *:before, *:after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
nav {
  margin: 1rem;
}
nav ul {
```

```
  list-style: none;
  border-bottom: solid 1px rgba(0,0,0, .3);
}
nav ul li {
  transition: all .5s;
  display: flex;
  justify-content: center;
  align-items: center;
}
nav ul li a {
  display: block;
  padding: 1rem;
  text-align: center;
  text-decoration: none;
  color: rgba(0, 0, 0, .8);
}
nav ul li:hover {
  background-color: #8E799C;
  box-shadow: 0 -8px 0 #5A3B6E inset;
}
nav ul li:hover a {
  color: white;
}
nav ul.proporcional {
  display: flex;
  flex-wrap: wrap;
}
nav ul.proporcional li {
  flex: 1 1 auto;
}
```

> Demo de menú de navegación con ítems de anchura proporcional mediante flexbox

Finalmente, si queremos que cada opción tenga la **anchura natural correspondiente a su texto y que los elementos flexibles no crezcan, sino que aparezcan centrados dejando todo el espacio sobrante a los lados**, tendremos que especificar para el contenedor flexible este tipo de justificación, mediante `justify-content: center` e indicar en los elementos flexibles que no deben de crecer, mediante `flex-grow: 0`. Veamos una demostración:

See the Pen Flexbox: Menú horizonal número indeterminado anchura natural centrado by Pedro Joya (@pedrojoya) on CodePen.

```
<nav>
  <ul class="natural">
    <li><a href="#" title="Inicio">Inicio</a></li>
    <li><a href="#" title="Blog">Blog</a></li>
    <li><a href="#" title="Trabajo">Trabajo y estudios varios</a></li>
    <li><a href="#" title="Recursos">Recursos</a></li>
    <li><a href="#" title="Acerca de">Acerca de</a></li>
  </ul>
</nav>
```

```
*, *:before, *:after {
  margin: 0;
```

```
  padding: 0;
  box-sizing: border-box;
}
nav {
  margin: 1rem;
}
nav ul {
  list-style: none;
  border-bottom: solid 1px rgba(0,0,0, .3);
}
nav ul li {
  transition: all .5s;
  display: flex;
  justify-content: center;
  align-items: center;
}
nav ul li a {
  display: block;
  padding: 1rem;
  text-align: center;
  text-decoration: none;
  color: rgba(0, 0, 0, .8);
}
nav ul li:hover {
    background-color: #8E799C;
  box-shadow: 0 -8px 0 #5A3B6E inset;
}
nav ul li:hover a {
  color: white;
}
nav ul.natural {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
}
nav ul.natural li {
  flex: 0 1 auto;
}
```

> Demo de menú de navegación centrado mediante flexbox

Menú en acordeón

Otro uso muy curioso de flexbox es crear **menús de tipo acordeón**, haciendo que **el elemento sobre el que nos situemos crezca mientras que el resto de elementos del menú decrezcan**. Veamos una demostración (basada en el siguiente enlace):

See the Pen Menú de acordeón con flexbox by Pedro Joya (@pedrojoya) on CodePen.

```
<ul class="acordeon">
  <li class="acordeon__celda fondo_rojo">
    <article class="acordeon__contenido">
      <h3 class="acordeon__titulo">Primero</h3>
```

```html
      <p class="acordeon__texto">Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam tristique eget augue eget gravida. Etiam eget imperdiet massa. Aliquam nisi eros, molestie a vulputate quis, tempor nec quam. Ut at libero in sem pellentesque imperdiet. Praesent pretium, sapien vel mollis porttitor, felis magna vehicula metus, ac dignissim odio tortor nec quam. Nam molestie, arcu id maximus lacinia, ante ipsum posuere mauris, ac maximus massa felis quis quam. Curabitur malesuada placerat nunc, id dapibus justo tempor in. Ut molestie arcu justo, id volutpat eros vestibulum eget. Fusce tincidunt leo in est venenatis porta.</p>
    </article>
  </li>
  <li class="acordeon__celda fondo_violeta">
    <article class="acordeon__contenido">
      <h3 class="acordeon__titulo">Segundo</h3>
      <p class="acordeon__texto">Aliquam nisi eros, molestie a vulputate quis, tempor nec quam. Ut at libero in sem pellentesque imperdiet. Praesent pretium, sapien vel mollis porttitor, felis magna vehicula metus, ac dignissim odio tortor nec quam. Nam molestie, arcu id maximus lacinia, ante ipsum posuere mauris, ac maximus massa felis quis quam. Curabitur malesuada placerat nunc, id dapibus justo tempor in. Ut molestie arcu justo, id volutpat eros vestibulum eget. Fusce tincidunt leo in est venenatis porta.</p>
    </article>
  </li>
  <li class="acordeon__celda fondo_azul">
    <article class="acordeon__contenido">
      <h3 class="acordeon__titulo">Tercero</h3>
      <p class="acordeon__texto">Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam tristique eget augue eget gravida. Etiam eget imperdiet massa. Aliquam nisi eros, molestie a vulputate quis, tempor nec quam. Ut at libero in sem pellentesque imperdiet. Curabitur malesuada placerat nunc, id dapibus justo tempor in. Ut molestie arcu justo, id volutpat eros vestibulum eget. Fusce tincidunt leo in est venenatis porta.</p>
    </article>
  </li>
  <li class="acordeon__celda fondo_verde">
    <article class="acordeon__contenido">
      <h3 class="acordeon__titulo">Cuarto</h3>
      <p class="acordeon__texto">Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nullam tristique eget augue eget gravida. Etiam eget imperdiet massa. Aliquam nisi eros, molestie a vulputate quis, tempor nec quam. Ut at libero in sem pellentesque imperdiet. Praesent pretium, sapien vel mollis porttitor, felis magna vehicula metus, ac dignissim odio tortor nec quam. Nam molestie, arcu id maximus lacinia, ante ipsum posuere mauris, ac maximus massa felis quis quam.</p>
    </article>
  </li>
</ul>
```

```css
.acordeon {
  display: flex;
  width: 100%;
  height: 350px;
  padding: 0;
  margin: 0;
  flex-wrap: row nowrap;
```

```
}
.acordeon__celda {
  padding: 1rem;
  /* Para que no se muestre lo que no cabe */
  overflow: hidden;
  /* Para que el efecto se haga poco a poco */
  transition: flex 0.3s;
  flex: 1 10;
}
.acordeon__celda:hover {
  /* Cuando nos colocamos encima crece ese elemento */
  flex: 10;
}
.fondo_rojo {
  background-color: #ef9a9a;
}
.fondo_violeta {
  background-color: #ce93d8;
}
.fondo_azul {
  background-color: #90caf9;
}
.fondo_verde {
  background-color: #a5d6a7;
}
.acordeon__contenido {
  /* Para que el texto no se vaya tanto para abajo */
  width: 100%;
  min-width: 400px;
}
```

> Demo de menú en acordeón

Otros ejemplos:

> https://www.codeseek.co/dermitderoboe/horizontal-flexbox-accordion-EWLPeB
>
> https://codepen.io/jcoulterdesign/pen/OMmZPd?editors=1100#0

Galería de imágenes

La dificultad real de esta técnica es conseguir establecer el ancho de los elementos atendiendo al número de columnas deseadas. Veamos una demostración:

See the Pen Flexbox: Galería de imágenes by Pedro Joya (@pedrojoya) on CodePen.

```html
<h1>Galería de imágenes</h1>
<div class="galeria">
  <img src="https://dummyimage.com/200x200/D4B56A/fff.png&text=Foto" alt="" />
  <img src="https://dummyimage.com/200x200/D4B56A/fff.png&text=Foto" alt="" />
  <img src="https://dummyimage.com/200x200/D4B56A/fff.png&text=Foto" alt="" />
  <img src="https://dummyimage.com/200x200/D4B56A/fff.png&text=Foto" alt="" />
  <img src="https://dummyimage.com/200x200/D4B56A/fff.png&text=Foto" alt="" />
  <img src="https://dummyimage.com/200x200/D4B56A/fff.png&text=Foto" alt="" />
  <img src="https://dummyimage.com/200x200/D4B56A/fff.png&text=Foto" alt="" />
  <img src="https://dummyimage.com/200x200/D4B56A/fff.png&text=Foto" alt="" />
</div>
```

```css
*, *:before, *:after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
html {
  font-family: Arial;
  --margen: .5rem;
  --columnas: 3;
  --anchura: calc((100% - (var(--columnas)-1)*var(--margen)) /var(--columnas));
}
h1 {
  margin: var(--margen) var(--margen) 0 var(--margen);
  text-align: center;
  padding: 1rem;
  background-color: #CCC;
}
.galeria {
  margin: 0 var(--margen) var(--margen) 0;
  display: flex;
  flex-wrap: wrap;
}
.galeria img {
  display: block;
  margin: var(--margen) 0  0 var(--margen);
  width: var(--anchura);
  height: var(--anchura);
}
```

> Demo de galería de imágenes

Pie pegajoso

Otro de los aspectos algo costosos era conseguir que el pie de la página estuviera situado siempre en la zona inferior si el contenido de la página tenía una altura pequeña. Gracias a *flexbox*, ahora es mucho más sencillo, **bastará con convertir el `<body>` en un contenedor flexible que disponga sus hijos en forma de columna. Para conseguir que el pie esté situado en la zona inferior haremos que el body tenga una altura mínima de 100vh (100% de la altura del *viewport*) y que el cuerpo pueda ocupe todo el espacio sobrante en el contenedor flexible.** Veamos una demostración:

See the Pen Flexbox: Pie siempre abajo by Pedro Joya (@pedrojoya) on CodePen.

```
<header>Soy la cabecera</header>
<main>
  <p>Lorem fistrum jarl de la pradera a wan no puedor apetecan quietooor. Está la cosa muy malar condemor amatomaa papaar papaar. No puedor no te digo trigo por no llamarte Rodrigor sexuarl te voy a borrar el cerito. Jarl de la pradera hasta luego Lucas está la cosa muy malar no te digo trigo por no llamarte Rodrigor papaar papaar al ataquerl ese pedazo de te va a hasé pupitaa</p>
</main>
<footer>Soy el pie</footer>
```

```
body {
  width: 80%;
  margin: 0 auto;
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
header {
  padding: 1rem;
  text-align: center;
  font-size: 1.5rem;
  background-color: #6A4E90;
  color: #DDD;
}
main {
  flex-grow: 1;
  background-color: #E1CBE3;
  padding: 1rem 2rem;
}
footer {
  padding: 1rem;
  font-size: 1.2rem;
  text-align: center;
  background-color: #555794;
  color: #DDD;
}
```

> Demo de pie pegajoso

El Santo Grial

En diseño web se conoce como "Santo Grial" (en inglés *Holy Grail*) al diseño característico de la web que muestra un encabezado en la zona superior, un pie en la zona inferior, y la zona central muestra tres columnas: el menú de navegación, el contenido principal y el contenido adicional (por ejemplo publicidad).
Gracias a *flexbox* podemos hacer este diseño de forma sencilla y además conseguir que se adapte a distintos tamaños de pantalla. Veamos una demostración:

See the Pen Flexbox: Holy grail by Pedro Joya (@pedrojoya) on CodePen.

```
<body class="holygrail">
  <header>Soy la cabecera</header>
  <div class="holygrail--body">
    <nav>Menú</nav>
```

```html
    <main>
      <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Assumenda odio optio, distinctio voluptatibus necessitatibus, sed enim, eveniet nostrum, laudantium tempore sit dolorem impedit. Nam inventore quia enim rem exercitationem a!</p>
      <p>Delectus rerum dolorum dignissimos incidunt iste aspernatur, velit neque dicta itaque quidem eius ut. Pariatur, nulla architecto nihil odio ad aperiam consectetur, cupiditate veritatis corporis inventore reiciendis soluta. Facilis, recusandae.</p>
      <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Consequuntur non autem beatae optio deserunt commodi laboriosam atque dicta minima nobis error odit, veritatis, vel excepturi illum amet ipsum enim, tempora.</p>
      <p>Soluta tempore voluptas totam modi odit, aspernatur perspiciatis cupiditate, molestias dolorum nisi porro, quas, vero nulla et! Rerum expedita dolorem fuga vitae nostrum odit adipisci minima, debitis voluptas esse ullam!</p>
    </main>
    <aside>Publicidad</aside>
  </div>
  <footer>Soy el pie</footer>
</body>
```

```css
*, *:before, *:after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
.holygrail {
  /* Para que el pie sea pegajoso */
  display: flex;
  flex-direction: column;
  min-height: 100vh;
}
.holygrail--body {
  /* Para que el pie sea pegajoso */
  flex-grow: 1;
  /* Dentro del cuerpo */
  display: flex;
  flex-wrap: wrap;
}
header {
  padding: 1rem;
  text-align: center;
  font-size: 1.5rem;
  background-color: #6A4E90;
  color: #DDD;
}
nav {
  flex: 1 1 10%;
  min-width: 150px;
  background-color: #F2E1EA;
  padding: 2rem;
}
main {
  background-color: #E1CBE3;
  padding: 2rem;
```

```
  flex: 10 1 62%;
  min-width: 480px;
}
aside {
  flex: 1 1 20%;
  min-width: 200px;
  background-color: #C3AFCE;
  padding: 2rem;
}
footer {
  padding: 1rem;
  font-size: 1.2rem;
  text-align: center;
  background-color: #555794;
  color: #DDD;
}
p {
  margin-bottom: 1rem;
}
```

> Demo de el santo grial

Huecos entre elementos

Un caso de alineación curiosa con los elementos flexibles es cuando aplicamos a uno de ellos un margen automático. Con ejemplo supongamos que tenemos un contenedor flexible que sitúa sus elementos en forma de fila, posicionados de izquierda a derecha, entonces si a uno de los hijos le damos la propiedad `margin-right: auto`, se dejará todo el espacio sobrante entre dicho elemento y el siguiente. Si, por el contrario le establecemos `margin-left: auto`, se dejará todo el espacio sobrante entre el anterior y dicho elemento. Veamos una demostración:

See the Pen Flexbox y margin auto by Pedro Joya (@pedrojoya) on CodePen.

```
<ul class="menu">
  <li class="menu__item">Item 1</li>
  <li class="menu__item">Item 2</li>
  <li class="menu__item">Item 3</li>
  <li class="menu__item">Item 4</li>
</ul>
```

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
ul {
  list-style: none;
}
body {
  background-color: #DEDEDE;
}
.menu {
  display: flex;
  max-width: 1140px;
```

```
  width: 80%;
  margin: 1rem auto;
  background-color: beige;
}
.menu__item {
  text-align: center;
  background-color: lightcoral;
  color: white;
  padding: 10px;
}
.menu__item:not(:first-child) {
  border-left: 1px solid;
}
/* El tercer item crea un hueco a su derecha */
.menu__item:nth-child(3) {
  margin-right: auto;
}
```

> Demo de menú de navegación con huecos entre los elementos mediante flexbox

Barra de navegación

En la siguiente demostración podemos ver cómo crear una barra de navegación típica de twitter haciendo uso de flexbox y del truco de los huecos entre elementos (basado en este ejemplo):

See the Pen Barra de navegación de twitter con flexbox by Pedro Joya (@pedrojoya) on CodePen.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Document</title>
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/font-
awesome/4.7.0/css/font-awesome.min.css">
</head>
<body>
  <ul class="twitter">
    <li><i class="fa fa-home"></i>Inicio</li>
    <li><i class="fa fa-bell-o"></i>Notificaciones</li>
    <li><i class="fa fa-envelope-o"></i>Mensajes</li>
    <li class="twitter__bird"><i class="fa fa-twitter"></i></li>
    <li><form><input placeholder="buscar en twitter"/><i class="fa fa-search">
</i></form></li>
    <li><i class="fa fa-user-circle-o"></i></li>
    <li><button>tweet</button></li>
  </ul>
</body>
</html>
```

```
ul {
  padding: 0;
  list-style: none;
  width: 90%;
```

```
  margin: 1rem auto;
  min-width: 550px;
}
.twitter {
  display: flex;
}
.twitter__bird {
  /* Empuja a los hermanos anteriores a la izquierda */
  margin-left: auto;
  /* Empuja a los hermanos posteriores a la derecha */
  margin-right: auto;
  color: rgba(85,172,238 ,1);
  font-size: 1.3em;
}
.twitter {
  border: 1px solid red;
  height: 46px;
  align-items: center;
  padding: 0 10px;
  border: 1px solid rgba(238,238,238 ,1);
  border-radius: 5px;
  color: rgba(117,117,117 ,1);
  box-shadow: 5px 10px 20px -20px rgba(85,172,238 ,1);
}
.twitter li {
  cursor: pointer;
}
.twitter li:hover{
  color: rgba(85,172,238 ,1);
}
.twitter li:not(:last-child):not(.twitter__bird) {
  margin-right: 10px;
}
.twitter li i:not(.fa-twitter):not(.fa-search):not(.fa-user-circle-o) {
  margin-right: 3px;
}
.twitter li > button {
  font-size: 0.8em;
  border: 0;
  background: rgba(85,172,238 ,1);
  color: #fff;
  border-radius: 100px;
}
```

Demo de barra de navegación de twitter

Referencias

https://developer.mozilla.org/es/docs/Web/CSS/CSS_Flexible_Box_Layout/Usando_las_cajas_flexibles_CSS

https://css-tricks.com/snippets/css/a-guide-to-flexbox/

http://codepen.io/starzonmyarmz/pen/WbGrgM

http://codepen.io/wadehammes/pen/EadRdw

> http://codepen.io/starzonmyarmz/pen/WbGrgM
>
> http://codepen.io/jbeason/pen/Wbaedb
>
> http://codepen.io/oknoblich/pen/klnjw
>
> https://philipwalton.github.io/solved-by-flexbox/
>
> https://speckyboy.com/css-flexbox-toolbox/
>
> http://www.flexboxpatterns.com/home
>
> http://www.unumux.com/ux-insights/2015/7/21/interface-patterns-using-flexbox
>
> http://codepen.io/terrymun/pen/cKxwu?editors=1100
>
> http://codepen.io/romamilkovic/pen/JYaQQe